

Лекция 7.

Централизованное резервное копирование и восстановление

Цель лекции

Понять backup как процесс создания, проверки и восстановления

Отличать backup от RAID, snapshot и replication

Настроить простой Borg backup с web01 на backup01

Создать две точки восстановления и проверить repository

Выполнить безопасный Test Restore в отдельный каталог

Учебные вопросы

1. RAID, Snapshot, Replication и Backup
2. Full, Incremental, Differential, 3-2-1, RPO и RTO
3. Backup Server и отдельная сервисная учетная запись
4. rsync, BorgBackup и краткий обзор Restic
5. Retention и удаление старых точек
6. Integrity Check и Test Restore

Главная идея: backup нужно восстановить

Backup ≠ создать файл копии

Backup = создать + проверить + восстановить

Source Data → Backup → Repository → Integrity Check →

Test Restore → Application Test

Если восстановление ни разу не проверяли, backup
остается предположением

Цель администратора - доказать, что данные можно
вернуть

Основная лабораторная топология

web01: 192.168.30.10

Источник данных: /srv/backup-source

backup01: 192.168.30.40

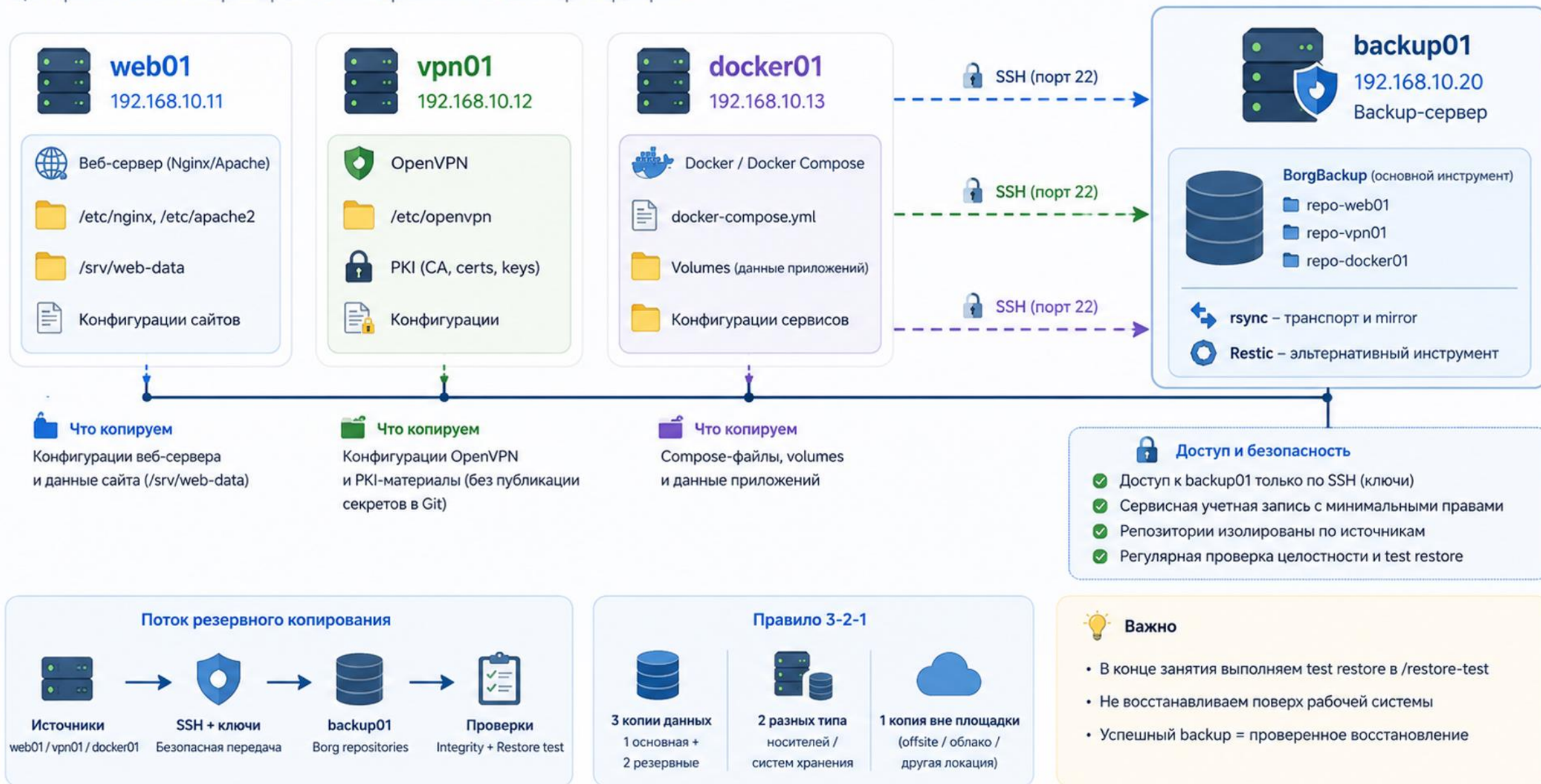
Borg repository: /backups/borg/web01

Связь между web01 и backup01 выполняется по SSH

vpn01 и docker01 рассматриваются позже как
масштабирование схемы

Топология: web01, vpn01, docker01 и backup01

Централизованное резервное копирование с backup-сервером



1. RAID, Snapshot и Replication не заменяют Backup

RAID помогает пережить отказ диска

Snapshot помогает быстро вернуться к предыдущему состоянию

Replication создает копию на другом узле

Backup создает независимую восстановительную точку

RAID ≠ Backup, Snapshot ≠ Backup, Replication ≠ Backup

Почему RAID не спасает от логических ошибок

Если пользователь удалил файл, RAID честно сохранит удаление

Если приложение испортило данные, RAID не вернет старую версию

Если шифровальщик изменил файлы, RAID сохранит измененные блоки

RAID нужен для доступности, но не для истории восстановления

Воскир нужен, чтобы вернуться к точке до ошибки

Snapshot и replication решают другие задачи

Snapshot часто находится рядом с исходными данными и зависит от той же системы хранения

Replication быстро переносит ошибку или удаление на второй узел

Snapshot удобен для короткого отката перед изменением

Replication полезна для отказоустойчивости

Backup должен быть независимой восстановительной точкой с проверкой restore

Схема: RAID, snapshot, replication и backup

Каждый механизм решает свою задачу. Полная защита достигается их совместным использованием.

1. RAID

Доступность дисковой подсистемы

RAID 1 / RAID 5 / RAID 10



Данные распределяются между несколькими дисками в одном массиве.

Что обеспечивает

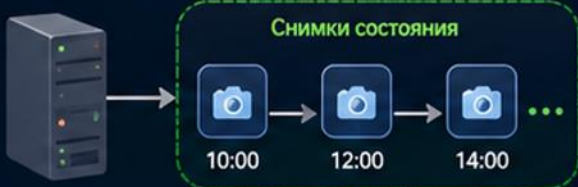
- ✓ Работу при отказе одного (или нескольких) дисков
- ✓ Высокую скорость чтения/записи
- ✓ Повышенную доступность данных

На что не защищает

- ✗ Удаление/изменение файлов пользователем
- ✗ Шифрование данных вредоносным ПО
- ✗ Сбои ОС, ошибки конфигурации, вирусы
- ✗ Пожар, кража, физическое уничтожение сервера

2. Snapshot

Зафиксированное состояние во времени



Фиксирует состояние тома/файловой системы на определённый момент.

Что обеспечивает

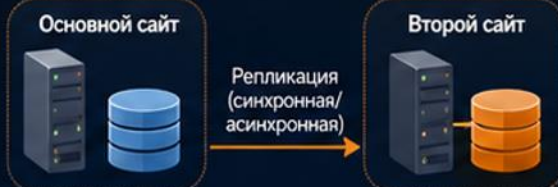
- ✓ Быстрый откат к предыдущему состоянию
- ✓ Удобно перед обновлениями и изменениями
- ✓ Минимальное влияние на производительность (на большинстве СХД)

На что не защищает

- ✗ Отказ хранилища, массива или сервера
- ✗ Удаление snapshot'ов злоумышленником
- ✗ Длительное хранение (обычно ограничено)
- ✗ Защиту от вымогателей без изоляции

3. Replication

Копия на другом узле или площадке



Передаёт изменения на другой сервер или в другой дата-центр.

Что обеспечивает

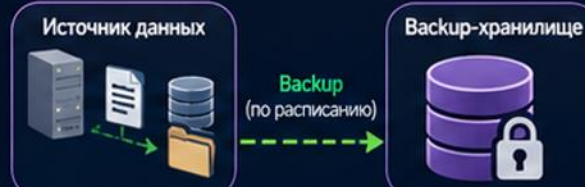
- ✓ Отказоустойчивость и быстрое переключение
- ✓ Работа сервиса при падении площадки
- ✓ Географическое распределение данных

На что не защищает

- ✗ Мгновенное копирование ошибок и удалений
- ✗ Вирусы и шифровальщики (копируются тоже)
- ✗ Долгосрочное хранение версий
- ✗ Случайные изменения конфигурации

4. Backup

Восстановительная точка



Создаёт копии данных, которые можно проверить и восстановить на другом узле или на том же.

Что обеспечивает

- ✓ Восстановление после любых логических сбоев
- ✓ История версий и долгосрочное хранение
- ✓ Возможность восстановления на «чистом» сервере
- ✓ Изоляцию (при правильной настройке)

На что не защищает

- ✗ Если копии доступны на запись злоумышленнику
- ✗ Если хранится на том же отказавшем ресурсе
- ✗ Без проверки копии могут оказаться «мусором»



Главный вывод

RAID, snapshot и replication повышают доступность и устойчивость, но только backup даёт настоящие восстановительные точки. Используйте их вместе по правилу 3-2-1.



RAID

Доступность

+



Snapshot

Быстрый откат

+



Replication

Отказоустойчивость

+



Backup

Восстановление

Правило 3-2-1



3

3 копии данных
1 основная + 2 резервные



2

2 разных типа носителей
например: диск + лента/облако



1

1 копия вне площадки
офсайт или облако

2. Full, Incremental и Differential описывают логику копий

Full - полный набор данных

Incremental - изменения после предыдущей точки

Differential - изменения после последнего Full

Эти термины помогают понять классические схемы резервного копирования

Borg и Restic используют дедупликацию, поэтому классическая схема выглядит иначе

Дедупликация уменьшает повторное хранение одинаковых данных

Borg сохраняет новые уникальные блоки данных

Если файл не изменился, его содержимое не хранится заново целиком

Студент видит несколько архивов, но repository использует место экономнее

Глубокий разбор chunk storage не нужен для первой лаборатории

Практический вывод: можно хранить несколько точек восстановления

Правило 3-2-1 задает минимальную устойчивость копий

3 экземпляра данных

2 разных типа или места хранения

1 копия вне основного риска

Пример: Production + Backup01 + Offsite copy

Если все копии находятся на одном сервере, общий риск остается слишком высоким

Дополнительно: 3-2-1-1-0 развивает классическое правило

+1 означает offline или immutable copy

0 означает проверенное восстановление без ошибок

Это развитие правила 3-2-1 для более зрелой эксплуатации

В junior-лаборатории главное - понять 3-2-1 и выполнить test restore

Immutable storage подробно рассматривается в production-блоках

RPO показывает допустимую потерю данных

RPO (Recovery Point Objective) - сколько данных допустимо потерять

RPO = 1 час означает, что точка восстановления должна быть не старше 1 часа

Расписание backup само по себе не гарантирует RPO

Если backup упал, фактический RPO становится хуже

Monitoring backup должен показывать возраст последней успешной копии

RTO показывает допустимое время восстановления

RTO (Recovery Time Objective) - сколько времени
можно восстанавливать сервис

RTO включает обнаружение аварии

RTO включает restore

RTO включает проверку данных и запуск сервиса

Если восстановление никогда не тестировали,
реальный RTO неизвестен

3. Backup Server хранит копии отдельно от источника

Backup Server - отдельный узел для хранения резервных копий

В лаборатории это backup01

Копия на том же диске не защищает от отказа этого диска

Копия на отдельном сервере снижает общий риск
backup01 тоже нужно защищать от переполнения и несанкционированного доступа

Push и Pull различаются направлением запуска

Push - источник отправляет backup на Backup Server

Pull - Backup Server забирает данные с источника

Для лаборатории используем Push, потому что он проще

Push легче понять на одном источнике web01

Security trade-offs Pull-архитектуры рассматриваются дополнительно

Backup Job должен иметь отдельную учетную запись

Используем пользователя backup или backup-web01

Не используем личную учетную запись администратора

Backup Job → отдельная учетная запись → минимальные права

Отдельная учетная запись упрощает аудит и ограничение доступа

Права должны позволять записывать backup, но не управлять всем сервером

SSH key настраивается простым проверяемым способом

`ssh-keygen -t ed25519` - создать ключ

`ssh-copy-id backup@backup01` - передать публичный ключ

`ssh backup@backup01` - проверить вход

`chmod 700 ~/.ssh` - закрыть каталог ключей

`chmod 600 ~/.ssh/id_ed25519` - закрыть приватный ключ

`forced command`, `from=`, `no-pty` и `wrapper scripts` -
дополнительно

SSH Host Key подтверждает сервер backup01

User Key подтверждает клиента

Host Key подтверждает сервер

Host Key защищает от подмены backup01

Не нужно советовать StrictHostKeyChecking=no

Fingerprint verification достаточно упомянуть как
проверку отпечатка сервера

4. rsync полезен как mirror, но не как история версий

rsync синхронизирует файлы между каталогами или узлами

Dry-run перед реальным запуском: `rsync -aHAXn /srv/backup-source/`

`backup@backup01:/backups/web01/current/`

После проверки: `rsync -aHAX /srv/backup-source/ backup@backup01:/backups/web01/current/`

-n означает dry-run

Для первой лаборатории rsync нужен как пример mirror

Завершающий slash в rsync меняет результат

`/source/` означает содержимое каталога `source`

`/source` означает сам каталог `source`

Эта разница часто ломает ожидаемую структуру копии

Перед автоматизацией результат нужно проверить в тестовом каталоге

`rsync --delete` не используем в первой лаборатории без отдельного `dry-run`

rsync mirror и Borg backup дают разные результаты

rsync current mirror показывает текущее состояние

Если файл удалили и синхронизировали mirror,
удаление попадет в current

Borg хранит историю архивов

Borg позволяет выбрать нужную точку восстановления

В этой лекции Borg является основным инструментом
лаборатории

Borg repository - место хранения архивов

```
export
```

```
BORG_REPO='ssh://backup@backup01/./backups/borg/  
web01'
```

```
borg init --encryption=repokey-blake2
```

Repository - место хранения backup

Encryption - repository хранится зашифрованным

Перед init нужно убедиться, что SSH-доступ к backup01 работает

Passphrase нельзя терять и нельзя хранить в Git

Passphrase защищает доступ к зашифрованному repository

Не хранить passphrase в Git

Не писать passphrase прямо внутри сценария

Не вставлять passphrase в публичную документацию

Потеряли passphrase или key - backup может стать недоступен

Borg key export переносится в дополнительный материал

Первый Borg backup создает первую точку восстановления

```
echo "Version 1" > /srv/backup-source/file.txt
```

```
borg create '::{hostname}-{now:%Y-%m-%d_%H-%M-%S}'  
/srv/backup-source
```

borg list - показать архивы в repository

borg info - показать информацию о repository или архиве

После первого backup студент должен увидеть первую точку восстановления

Второй Borg backup показывает историю изменений

```
echo "Version 2" > /srv/backup-source/file.txt
```

Повторно выполнить `borg create`

`borg list` должен показать две точки восстановления

Студент должен понимать: это не две полные копии в классическом смысле

Цель - увидеть, что можно выбирать нужный архив для восстановления

Break/Fix: удаленный файл нужно восстановить из нужной точки

```
rm /srv/backup-source/file.txt
```

Симптом: важный файл исчез из источника

Задача: восстановить Version 1 из первого архива

Нельзя сразу восстанавливать поверх live-каталога

Сначала выполняется Test Restore в отдельный каталог

Test Restore выполняется в отдельный каталог

```
mkdir -p /restore-test/web01
```

```
cd /restore-test/web01
```

```
borg extract '::archive-name'
```

```
cat srv/backup-source/file.txt
```

Ожидаемый результат: Version 1

Так студент доказывает, что архив можно реально
прочитать и восстановить

Restore сначала проверяют, потом переносят в live

Restore → сначала /restore-test

verify → проверить содержимое

copy to live → переносить только после проверки

Проверить права и владельца

Проверить конфигурацию или данные приложения

Так снижается риск испортить рабочую систему

неверным restore

Перед backup нужно проверить, что источник смонтирован

```
mountpoint -q /srv/data
```

```
findmnt /srv/data
```

Если filesystem не смонтирован, каталог может быть пустым

Backup может завершиться с exit code 0, но данных в копии не будет

Source Mount Check - обязательная проверка для каталогов на отдельных файловых системах

Консистентность приложения важнее слепого копирования файлов

Обычные файлы можно копировать напрямую

Database лучше сохранять через dump или специальный backup

Примеры: mysqldump и pg_dump

pg_basebackup, quiesce и Docker internals не входят в основную часть лекции

Главная идея: восстановленный сервис должен работать, а не просто иметь файлы

Restic рассматриваем только обзорно

Restic - альтернативный backup tool

Restic поддерживает encryption

Restic работает со snapshots

Restic поддерживает разные backend

Пример: `restic backup /srv/backup-source`

В этой лекции Restic не становится второй полноценной лабораторией

5. Retention определяет, сколько точек хранить

Retention - политика хранения точек восстановления

Пример: 7 daily, 4 weekly, 6 monthly

```
borg prune --keep-daily=7 --keep-weekly=4 --keep-monthly=6
```

Retention должен соответствовать RPO, объему данных и месту на backup01

В первой лаборатории не углубляемся в locking и concurrency

Excludes нужно проверять восстановлением

cache можно исключать

tmp можно исключать

generated files можно исключать

Каждый exclude нужно проверять через restore

Не все журналы являются мусором: часть logs может быть нужна для расследования инцидента

Бэкап должен запускаться автоматически

Расписание делает бэкап регулярным

Для junior достаточно понять идею systemd timer

OnCalendar=daily

Persistent=true

RandomizedDelaySec, Nice и IOSchedulingClass

переносим в production-материал

Дополнительно: backup jobs не должны запускаться параллельно

Если предыдущий backup еще идет, новый запуск может помешать repository

Для защиты от параллельного запуска можно использовать flock

В базовой лаборатории достаточно знать сам риск

Подробная настройка locking переносится в дополнительные материалы

Главный фокус остается на create, check и restore

6. Integrity Check проверяет repository технически

borg check проверяет, что repository технически читается

borg check помогает найти повреждения backup repository

borg check ≠ сервис восстановлен

Integrity Check не заменяет Test Restore

После check нужно выполнить восстановление хотя бы в тестовый каталог

Диаграмма: цикл backup, проверка и restore

Надёжность достигается не созданием копии, а регулярной проверкой и реальным восстановлением



Три уровня проверки backup

1. Backup Job Success - команда завершилась без ошибки
 2. Repository Integrity - borg check подтверждает читаемость repository
 3. Test Restore - данные восстановлены в отдельный каталог
 4. Application Test - сервис или данные реально работают
- Exit code 0 - только первый уровень, а не доказательство полноценного восстановления

Backup monitoring связывают с RPO

Проверять последний успешный backup

Проверять размер свежей копии

Проверять возраст последней точки

Проверять наличие ожидаемых файлов

Backup age напрямую связан с RPO

Восстанавливают сервис, а не просто файлы

Работающий сервис = config + data + packages + permissions + secrets

Одних файлов может быть недостаточно

После restore нужно проверить приложение

SELinux, AppArmor, firewall и DNS не разбираем подробно в этой лекции

Главная мысль: успешный cat файла не всегда означает восстановленный сервис

Основной лабораторный цикл: backup

Создать service account

Создать SSH key

Проверить SSH к backup01

Инициализировать Borg repository

Создать Version 1

Сделать Backup #1

Изменить файл на Version 2

Сделать Backup #2

Основной лабораторный цикл: restore

Посмотреть borg list

Удалить файл

Restore Backup #1 в /restore-test

Проверить содержимое

Выполнить borg check

Настроить простой retention

Объяснить RPO и RTO

Записать вывод: backup работает, потому что restore проверен

Дополнительный production-материал

3-2-1-1-0, immutable storage и quotas

forced command, from=, no-pty и sudo wrapper

Borg key export и отдельные accounts на каждый
ИСТОЧНИК

Pull architecture и advanced systemd timer

Restic CLI, database quiesce, flock и repository locking

Эти темы важны, но не должны закрывать junior-цель:
backup + check + test restore

Итоги лекции

Backup отличается от RAID, snapshot и replication тем, что дает независимую точку восстановления

RPO отвечает за допустимую потерю данных, RTO - за допустимое время восстановления

Borg является основным инструментом лаборатории, rsync показан как mirror, Restic - обзорно

Backup должен иметь service account, SSH-доступ и проверяемое хранилище

borg check проверяет repository, но только Test Restore доказывает восстановление

Junior должен уметь создать копию. выбрать версию.

Контрольные вопросы

1. Почему RAID не является Backup?
2. Чем Snapshot отличается от Backup?
3. Что означает RPO?
4. Что означает RTO?
5. Для чего нужен отдельный Backup Server?
6. Чем rsync mirror отличается от Borg Backup?
7. Что такое Retention?
8. Почему exit code 0 не гарантирует хороший Backup?
9. Почему Restore выполняют сначала в отдельный каталог?
10. Почему Test Restore обязателен?